



LUDIKSCIENCES

Qu'est-ce que le BLE ?

Le Bluetooth Low Energy (**BLE**) est une technologie de communication sans fil conçue pour :

- consommer très peu d'énergie
- transmettre de petites quantités de données
- fonctionner en proximité

Contrairement au Wi-Fi ou au Bluetooth classique, le BLE est optimisé pour les objets connectés (**IoT**).

Exemples d'utilisation :

- montre connectée
- capteur de température
- badge d'accès
- tracker (AirTag, etc.)
- objets domotiques



BLE vs Bluetooth

	BLE	Bluetooth classique	
Consommation	Très faible ~1 à 15 mA (pics courts)	Élevée ~30 à 100 mA	5 à 20x moins
Débit	Faible ~10 à 100 kbit/s	Moyen ~1 à 2 Mbit/s	10 à 100x moins
Usage	Capteurs, IoT	Audio, périphériques	
Connexion	Rapide ~3 à 10 ms	Plus lourde ~100 ms à 1 s	100x plus rapide

Exemples d'utilisation Bluetooth classique :

- audio (casques)
- souris/clavier

BLE peut « dormir » entre deux échanges → économie d'énergie



Les rôles BLE

Peripheral (serveur)

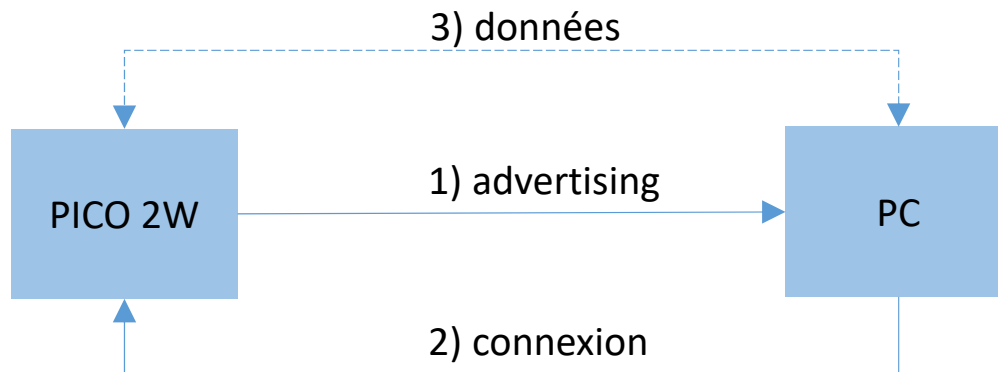
diffuse sa présence (advertising)
expose des données
attend une connexion

Exemple :
Pico 2 W
capteur

Central (client)

scanne les périphériques
se connecte
lit / écrit les données

Exemple :
PC
smartphone



L'**advertising** est le mécanisme par lequel un périphérique BLE :

- annonce sa présence
- diffuse son nom
- indique ses services



Différentes couches du protocole

La couche **GAP** (Generic Access Profile) est chargée de rendre l'objet visible. Elle gère la découverte de l'objet et l'établissement des connexions.

La couche **GATT** (Generic Attribute Protocol) rassemble les attributs en services et gère les échanges entre les objets : le client adresse ses requêtes au serveur GATT et peut lire/écrire des données dans le serveur

```
Service
├── Characteristic
│   ├── valeur
│   └── read / write / notify
```

En pratique :

```
Service: Température
├── Characteristic: valeur température
```

Advertising → Scan → Connexion → GATT → Échanges

READ (lecture) : PC → Pico : "donne-moi la température"

WRITE (écriture) : PC → Pico : "allume la LED"

NOTIFY (notification) : Pico → PC : "TEMP=23.5"



Premiers essais

Mise en marche côté Pico :

```
import bluetooth
import time

ble = bluetooth.BLE()
ble.active(True)

print("BLE actif")

while True:
    time.sleep(1)
```

Que fait ble.active(True) ?
Le Pico est-il visible en BLE ?



Advertising

Sur le Pico :

Que signifie "Pico2W" ?
À quoi sert gap_advertise() ?

```
import bluetooth
import struct
import time

ble = bluetooth.BLE()
ble.active(True)

def advertising_payload(name):
    payload = bytearray()
    # Flags: general discoverable + BR/EDR not supported
    payload += struct.pack("BBB", 2, 0x01, 0x06)
    # Complete Local Name
    name_bytes = name.encode()
    payload += struct.pack("BB", len(name_bytes) + 1, 0x09)
    payload += name_bytes

    return payload

payload = advertising_payload("Pico2W")
ble.gap_advertise(100000, adv_data=payload, connectable=True)

print("Advertising...")
print(payload)

while True:
    time.sleep(1)
```



Scanner depuis Ubuntu

Sur le PC :

```
$ pip install bleak
```

```
import asyncio
from bleak import BleakScanner

async def main():
    devices = await BleakScanner.discover()

    for d in devices:
        print(d.name, d.address)

asyncio.run(main())
```

=> On doit trouver une ligne de type : **Pico2W** 88:A2:9E:32:02:C7



LUDIKSCIENCES

Travaux pratiques

TP 1 — PC vers Pico

Nous utiliserons sur Ubuntu : `pc_bleak_client_to_pico.py`

Et sur pico : `pico_ble_peripheral.py`

Tester les fonctions depuis le PC : PING / LED ON / LED OFF / STATUS

Comment ça marche ?

Côté PC `await client.write_gatt_char(RX_UUID, b"PING")`

Côté Pico

```
# réception de données
if event == 3:
    # write
    data = ble.gatts_read(rx_handle)
    print("Reçu:", data)
```

Réponse:

Côté Pico `ble.gatts_write(tx_handle, b"PONG")`

Côté PC

```
value = await client.read_gatt_char(TX_UUID)
print(value)
```



Travaux pratiques

Comment ça marche ?

1) Du PC vers le pico

Côté PC `await client.write_gatt_char(RX_UUID, b"PING")`

Côté Pico

```
# réception de données
if event == 3:
    # write
    data = ble.gatts_read(rx_handle)
    print("Reçu:", data)
```

2) Du Pico vers le PC - Réponse

Côté Pico `ble.gatts_write(tx_handle, b"PONG")`

Côté PC

```
value = await client.read_gatt_char(TX_UUID)
print(value)
```

3) Notifications

Côté Pico `ble.gatts_notify(conn_handle, tx_handle)`

Côté PC `await client.start_notify(TX_UUID, handler)`

Différence read vs notify ?

- read = à la demande
- notify = automatique



Travaux pratiques

TP 2 — Exemple lot

Le Pico 2 W envoie toutes les secondes une chaîne du type :
TEMP=23.41

Le PC :

se connecte au Pico

- s'abonne aux notifications
- extrait la température
- met à jour un graphique en direct

Nous utiliserons les fichiers :

- Pico : `pico_ble_temp.py`
- PC : `pc_ble_graph_temp.py`



Observations

Si on coupe le programme PC, le pico n'émet plus de données !

- 1) Pourquoi le graphique ne commence-t-il à bouger qu'après la connexion BLE ?
- 2) Pourquoi garde-t-on seulement les MAX_POINTS dernières valeurs ?
- 3) Quel est l'intérêt de deque(maxlen=...) ?
- 4) Pourquoi a-t-on séparé :
 - la partie BLE
 - la partie graphique ?
- 5) Quel est le rôle de start_notify() ?
- 6) Pourquoi utilise-t-on plt.pause(0.1) ?
- 7) Que se passe-t-il si le Pico arrête d'envoyer des valeurs ?
- 8) Que se passe-t-il si on ferme la fenêtre de matplotlib ?



Réponses

- 1) Le graphique attend des données réelles. Sans notification pas de tracé.
- 2) Pour éviter :
 - une consommation mémoire croissante
 - un affichage trop chargé
 - une courbe illisible sur une longue durée
- 3) `deque(maxlen=...)` conserve automatiquement une fenêtre glissante des dernières mesures.
- 4) Le BLE et l'affichage n'ont pas le même rythme. Les séparer rend le programme plus lisible et stable.
- 5) `start_notify()` active la réception automatique des notifications sur la caractéristique choisie.
- 6) `plt.pause()` permet de faire tourner la boucle d'événements graphique et de rafraîchir la figure ; la documentation Matplotlib le présente comme une solution simple pour une animation rudimentaire.
- 7) La courbe reste figée sur les dernières valeurs reçues.
- 8) Le programme principal s'arrête. On l'arrête aussi au clavier (CTRL+C).

